

Building Advanced AI Agents with the LangChain Family

PART 1 — LangGraph & LangChain Complete Mastery

Instructor Teaching Script • v5.0 FINAL • 7 Hours Content • 2025

Audience	Developers who call LLM APIs and bind basic tools — building on top
Notebook	59 cells • 49 code + 10 markdown • 2,765 lines of runnable code
Free stack	Groq (Llama 3.3) • Tavily • LangSmith — zero credit card required
This PDF	WHY and HOW and WHEN — narrative, mental models, instructor notes
Part 2	Every concept here applied in a live 5-agent email pipeline

Read this before running the notebook. Understand first. Execute second.

The 8-Hour Promise

By the end of this two-part programme, you will have built a production-ready 5-agent pipeline that researches a topic on the web, writes a structured report, has that report reviewed by an LLM judge, seeks human approval, then formats and delivers the final result as a real HTML email. You will have full LangSmith observability into every step, the ability to pause before the email is sent for human review, and the ability to replay any part of the pipeline from any past checkpoint.

That is Part 2. Part 1 — the next 4 hours — gives you every tool, pattern, and mental model you need to build that system. Nothing in Part 2 will require you to learn something new. Every technique is introduced, practised, and understood here first.

Why the LangChain Ecosystem Exists

Building agents with raw LLM APIs is like building a web server with raw sockets. You can do it, but you end up reinventing the same things: message management, tool dispatch, conversation history, retry logic, streaming, debugging, evaluation. The LangChain family solves all of these, and it solves them in a way that composes cleanly — each layer builds on the one below.

langchain-core	The vocabulary. Messages, Runnables, Tools, Prompts.
■■ langchain	Composition. LCEL chains, the pipe operator.
■■ langchain-openai	LLM adapters. ChatOpenAI works with any provider.
■■ langchain-community	Tools, integrations, Tavily search.
langgraph	Orchestration. Graphs, state, checkpointing, HITL.
■■ .graph	StateGraph, START, END, Send, Command
■■ .checkpoint	MemorySaver, SqliteSaver, PostgresSaver
■■ .prebuilt	create_react_agent, ToolNode
langsmith	Observability + Evaluation + Prompt Management
■■ tracing	Automatic. Every call traced with zero code changes.
■■ hub	Push/pull versioned prompts like git commits.
■■ evaluation	Datasets, evaluators, experiment comparison.

The key insight: these packages have a clear job hierarchy. langchain-core is the shared vocabulary everyone speaks. LangChain adds composition — the ability to chain components together. LangGraph adds orchestration — stateful, persistent, resumable agent graphs. LangSmith adds visibility — you see everything, measure everything, improve everything. Mastering all four is what this programme delivers.

➡■■ We start in Section 0 by solving the first practical problem: provider lock-in. You will learn a pattern that makes your code work with Groq, Together AI, Mistral, Ollama, and a dozen other providers by changing exactly one line.

SECTION 0

Environment Setup

0:00 – 0:20 • 20 minutes • Cells 0.1 – 0.3

■ 0:00–0:20 • 20 min • Cells 0.1–0.3

The Provider Lock-In Problem — and How We Solve It on Day One

Most LangChain tutorials start with an OpenAI API key. This creates an invisible dependency that is easy to miss until you need to change it. When you want to switch to Groq for 10x faster inference, or to Ollama for local private inference, or to Together AI for different pricing, you discover that every example in that tutorial assumed OpenAI.

We solve this in Cell 0.3 with the provider-portability pattern. The OpenAI API format has become an industry standard. Groq, Together AI, Mistral, Ollama, OpenRouter, AWS Bedrock, DeepInfra, Anyscale — all of them expose an OpenAI-compatible endpoint. ChatOpenAI is not an OpenAI-specific class; it is a class that speaks the OpenAI API format, and it works with any provider that does the same.

The Provider-Portability Pattern

```
llm = ChatOpenAI(  
    model = "llama-3.3-70b-versatile",  
    api_key = os.environ['GROQ_API_KEY'],  
    base_url = "https://api.groq.com/openai/v1", # <-- only Groq-specific line  
    temperature=0.1, timeout=30, max_retries=2,  
)  
  
# To switch to Together AI: change base_url + api_key + model.  
# Every graph, tool, and prompt you build stays identical.
```

■ ■ Key concept

LangSmith activates automatically when you set two env vars:

LANGCHAIN_TRACING_V2 = 'true'

LANGCHAIN_API_KEY = your key

Every LangChain/LangGraph call from that point is traced. Zero other code changes.

➡■ With the environment ready, we move to the foundation of everything: Messages. Before touching graphs or tools, you need a deep understanding of the 5 message types — because every bug you will ever see in a LangChain application traces back to a misunderstanding of messages.

SECTION 1

Ecosystem + Messages + Why LangGraph

0:20 – 0:45 • 25 minutes • Cells 1.1 – 1.4

■ 0:20–0:45 • 25 min • Cells 1.1–1.4

Everything Is a Message

In LangChain, every piece of data that flows between you and a language model is a Message. Not a string, not a dict — a typed Message object. This might feel like extra ceremony at first, but it is one of the most important design decisions in the library. It means every component that processes messages can be composed with every other, it means your conversation history has structure, and it means LangSmith can display every message in a clear trace.

Type	When It Appears	Critical Detail
HumanMessage	User input to the LLM	Has name field to identify the speaker
AIMessage	LLM response	content may be empty string when tool_calls is populated
SystemMessage	Framing instructions	NEVER stored in AgentState — prepended fresh each node call
ToolMessage	Result of a tool execution	MUST include tool_call_id matching the AIMessage entry
FunctionMessage	Legacy (pre-2023)	Avoid in new code — use ToolMessage instead

The AIMessage: More Complex Than It Looks

AIMessage deserves special attention because it has two very different forms. When the LLM is answering a question, content is a string. When the LLM is calling a tool, content is an empty string and tool_calls is a list of tool call objects. Your router functions check for this distinction constantly.

AIMessage also carries usage_metadata — a dict with input_tokens and output_tokens. This is the foundation of cost tracking. In production, you accumulate these values across all LLM calls in a pipeline run to calculate total cost.

■ Bug to avoid

The ToolMessage tool_call_id must exactly match the id in the corresponding AIMessage.tool_calls entry. A mismatch is a silent bug — the LLM cannot link the result to the call it made. Always construct ToolMessage with: tool_call_id=tc['id']

The Manual Loop: Building the Case for LangGraph

Cell 1.3 asks you to build the manual agent loop — the while-loop pattern you probably already know. We do this deliberately: not because it is the right approach, but because you need to feel its limitations before you can appreciate why LangGraph exists.

Limitation	What Actually Breaks
Conditional routing	3+ branches becomes unmaintainable nested if/else chains
Persistent memory	State dies with the function. Nothing survives across calls.
Human-in-the-Loop	The loop has no pause concept. You cannot wait for a human.
Multi-agent handoff	You need an entirely new architecture to delegate to Agent B.
Parallel tools	The loop is sequential. Two tools cannot run simultaneously.
Crash recovery	A crash at step 9 means starting over from step 1.
Structured debugging	You only have print(). No structured trace, no history.

■ Instructor voice

LangGraph solves all 7 of these using the same tool-calling you already know. The mental model shift is simple:

BEFORE: agent = while loop (you manage everything)

AFTER: agent = directed graph (LangGraph manages execution)

➡■ Now that you understand why the manual loop is the wrong abstraction, we build the right one. Section 2 introduces the 4 primitives that replace the while loop with something that scales, persists, streams, and debugs cleanly.

SECTION 2

The 4 LangGraph Primitives

0:45 – 1:25 • 40 minutes • Cells 2.1 – 2.8

0:45–1:25 • 40 min • Cells 2.1–2.8

The Mental Model: From Loop to Graph

Every LangGraph application — from a two-node prototype to a 20-agent production system — is built from exactly 4 primitives. Master these 4 and you can build anything. The analogy that helps most students: imagine a meeting room with a whiteboard. The whiteboard (State) is visible to everyone. Each person (Node) reads what is on the whiteboard, does their work, and updates it. The hallways between rooms (Edges) determine who goes next. The building blueprint (Graph) shows the whole layout.

Primitive	What It Is	The Analogy
State	TypedDict shared by all nodes. Fields have reducers that define how it changes	The whiteboard in the meeting room
Nodes	Python functions: State in → update dict out. Never read from State	Attendees, each taking their part, updating the whiteboard
Edges	Normal (always A→B) or Conditional (call a function to decide)	Roads. Conditional edges are traffic lights with logic.
Graph	StateGraph container. Compile it to get a runnable.	The organisation chart — assembled once, run many times.

Primitive 1: State — The add_messages Reducer Is Everything

The most common LangGraph beginner mistake is forgetting the add_messages annotation on the messages field. Without it, every time a node returns a new message, it REPLACES the entire messages list. Your conversation history is erased on every step. This is not an obvious bug — the graph runs fine, it just has no memory of anything that happened before the last node.

WITHOUT add_messages (wrong):	WITH add_messages (correct):
State: [Human, AI_1]	State: [Human, AI_1]
Node returns: [AI_2]	Node returns: [AI_2]
State becomes: [AI_2]	State becomes: [Human, AI_1, AI_2]
X History ERASED every step	✓ History GROWS as conversation continues

Beyond add_messages, you can annotate any field with a custom reducer. operator.add on an integer field accumulates values: each node that returns {'search_count': 1} adds 1 to the existing count. operator.add on a list field concatenates: each node appends to the existing list. Fields with no reducer use last-write-wins: the latest return value replaces the field.

Primitive 2: Nodes — Four Patterns

A node is a Python function. It receives the current State and returns a dict containing only the fields that changed. LangGraph merges that dict into the existing state using each field's reducer. Four patterns cover almost every real use case:

- **Pattern 1 — Basic LLM node:** call `llm.invoke(state['messages'])`, return `{'messages': [response]}`
- **Pattern 2 — Dynamic system prompt:** build `SystemMessage` from state values, prepend it to messages, call LLM. The `SystemMessage` is NOT stored in state.
- **Pattern 3 — Multi-field update:** return a dict updating multiple fields simultaneously. All reducers apply at once.
- **Pattern 4 — ChatPromptTemplate node:** define the template outside the node, call `format_messages()` inside. Reusable, testable, composable.

Primitive 3: Edges — Routing Is Just Python

A conditional edge function is a plain Python function that receives the current State and returns a string naming the next node. Any logic is valid: check message content, compare counters, evaluate business rules. The most important practice: unit-test your router functions BEFORE wiring them into the graph. A router is called at every single node completion. A bug there breaks all routing silently. Tests cost nothing and catch everything.

Primitive 4: Graph Assembly — The Universal Pattern

Every LangGraph application follows the same 6-step assembly pattern. It never changes. Once you have it in muscle memory, building a new graph takes minutes.

The Universal Graph Assembly Pattern

```
builder = StateGraph(MyState) # 1. bind to State
builder.add_node('llm', llm_node) # 2. register nodes
builder.add_node('tools', ToolNode([...]))
builder.add_edge(START, 'llm') # 3. entry point
builder.add_conditional_edges( # 4. routing
    'llm', router_fn, {'tools':'tools', '__end__':END})
builder.add_edge('tools', 'llm') # 5. loop back
graph = builder.compile() # 6. compile
display(Image(graph.get_graph().draw_mermaid_png())) # ALWAYS visualise
```

Common mistake

Always visualise with `draw_mermaid_png()` immediately after `compile()`.
 The diagram catches structural bugs (missing edges, wrong loops)
 that are invisible in the code but obvious in the picture.

All 4 Streaming Modes

Once your graph is compiled, you choose how to observe its execution. The mode you choose depends on what you need to see and who is consuming the output.

Mode	What It Yields	When to Use It
<code>stream_mode='updates'</code>	State delta after each node fires	Development: see exactly what each node changed
<code>stream_mode='values'</code>	Full state snapshot after each node	Audit trails, full-state replay
<code>stream_mode='messages'</code>	New messages only, as they appear	Chat UI: stream conversation turns to the user

<code>astream_events(version='v2')</code>	Every LLM token + every tool start/end	Real-time streaming to a web UI — the ChatGPT experience
---	--	--

update_state(): The Power to Correct Running Graphs

`update_state()` lets you inject or correct state from OUTSIDE the graph while it is paused at a checkpoint. This is not an edge case — it is the foundation of Human-in-the-Loop workflows: the graph runs until a HITL interrupt, a human reads the state, calls `update_state()` to correct a field, then resumes. We cover the full HITL flow in Section 8. Cell 2.8 introduces the mechanics so the concept is not new when you need it.

➡■ You now have the complete graph machinery. The next step is to master the components that go INSIDE the nodes: professional prompt construction with `ChatPromptTemplate`, and composable chains with `LCEL`. Section 3 covers both.

SECTION 3

LCEL + Prompt Engineering

1:25 – 2:00 • 35 minutes • Cells 3.1 – 3.5

■ 1:25–2:00 • 35 min • Cells 3.1–3.5

Why f-Strings Are the Wrong Tool for Prompts

When you write a prompt as an f-string, it works. But then you need to reuse it in a different node with different variables, so you copy-paste it and change a few words. Then you need to add conversation history to it, so you concatenate a list of strings. Then you want to test it in isolation, but it only works when you have a live state object. Within weeks, your prompts are scattered, duplicated, untestable, and fragile.

ChatPromptTemplate exists to solve all of these problems at once. It is a typed, reusable, composable prompt that can be defined once, used anywhere, tested independently, and composed with the pipe operator to form a complete chain.

ChatPromptTemplate: Typed, Reusable, Composable

ChatPromptTemplate: Definition and Usage

```
template = ChatPromptTemplate.from_messages([
    ('system', 'You are a {role} expert in {domain}. Reply in {language}.'),
    ('human', '{question}'),
])
# Use as a chain: template | llm
chain = template | llm
result = chain.invoke({'role': 'analyst', 'domain': 'AI', 'language': 'English', 'question': '...'})
# Or get the compiled messages without calling the LLM:
msgs = template.format_messages(role='analyst', domain='AI', language='English', question='...')
```

MessagesPlaceholder: Injecting Conversation History

MessagesPlaceholder is one of the most important tools in LangGraph development. It reserves a named position in a template for a full list of messages. This is the correct way to inject conversation history into a prompt — not by concatenating strings, not by looping over messages and formatting them, but by placing a MessagesPlaceholder in the template and passing the messages list at invoke time.

MessagesPlaceholder in a LangGraph Node

```
agent_prompt = ChatPromptTemplate.from_messages([
    ('system', 'Topic: {topic}. Searches done: {count}.',
    MessagesPlaceholder(variable_name='messages'), # history injected here
])
# Inside a LangGraph node:
def research_node(state):
    compiled = agent_prompt.format_messages(
        topic = state['research_topic'],
        count = state['search_count'],
        messages= list(state['messages']) # full history injected
    )
```

```
return {'messages': [llm.invoke(compiled)]}
```

The `partial()` method lets you fix some variables and leave others open. This is how you create multiple specialised agents from one base template: `english_analyst = base_template.partial(language='English', role='analyst')`. Now `english_analyst` only needs domain and question at invoke time.

partial(): One Template, Many Agents

The `partial()` method is how you create a family of specialised agents from one base template. Instead of writing five similar prompts with minor variations, you write one base template and use `partial()` to lock in the variables that do not change. The resulting partial template only needs the remaining variables at invoke time.

In Part 2, you will see this pattern for the Researcher: one base prompt template, partially applied with a fixed `research_topic`, then invoked with fresh messages from state on every call. In production, `partial()` is how you create tenant-specific agents where each tenant has a different language, tone, or domain expertise without duplicating the base prompt logic.

LCEL: The Pipe Operator Makes Chains Composable

LCEL (LangChain Expression Language) is the composition system that connects templates, LLMs, tools, and parsers using the familiar Unix pipe operator `|`. Every LangChain component is a Runnable. Runnables have a contract: they take an input and return an output. Chaining them with `|` passes the output of each as the input to the next.

But LCEL is not just syntactic sugar for function composition. A chain built with `|` gets streaming for free (each component streams if it can), batching for free (invoke a chain on 100 inputs in parallel), async for free (every chain has an `ainvoke()` and `astream()`), and LangSmith tracing for free (every chain step is a span).

Runnable	What It Does	Common Usage
ChatPromptTemplate	Builds typed message list from variables	template llm
LLM (ChatOpenAI)	Calls the language model	... llm parser
StrOutputParser	Extracts .content → plain string	... llm StrOutputParser()
JsonOutputParser	Parses JSON text → Python dict	... llm JsonOutputParser()
PydanticOutputParser	Parses + validates → typed object	... llm PydanticOutputParser(Model)
RunnableLambda	Wraps any Python function	RunnableLambda(my_fn)
RunnablePassthrough	Passes input + adds new fields	RunnablePassthrough.assign(field=fn)
RunnableParallel	Runs multiple chains simultaneously	{'a':chain1, 'b':chain2}
RunnableBranch	Conditional routing in a chain	RunnableBranch((cond,chain), default)

■ Design principle

RunnableParallel runs ALL branches with the SAME input simultaneously.
Three LLM calls in a RunnableParallel = roughly 1x latency, not 3x.
Use it whenever you need multiple independent results from one input.

■ Key concept

INSTRUCTOR: The Researcher node (Cell 3.2) takes 12-15 min to teach, not 8.
Students need to understand the internal loop before the bind_tools fix lands.
Run Cell 3.6 (isolation test) immediately after 3.5 to build confidence.
Most common question: 'Why do I need bind_tools?' -> show them without it.

➡■ LCEL gives you the composition layer. Now you need the building blocks that go into the chain: tools (how agents act on the world), output parsers (how agents produce structured data), callbacks (how you observe everything), and async (how you serve many users). Section 4 covers all four.

SECTION 4

Tools + Parsers + Callbacks + Async

2:00 – 2:35 • 35 minutes • Cells 4.1 – 4.6

■ 2:00–2:35 • 35 min • Cells 4.1–4.6

Tools: How Agents Act on the World

A tool is a Python function that the LLM can request to call. The `@tool` decorator is the production standard for creating them. It takes a Pydantic model as the `args_schema`, which serves two purposes simultaneously: the LLM receives a typed JSON Schema so it can construct valid arguments, and Pydantic validates those arguments before your function runs.

The docstring on a `@tool` function is not documentation for humans. It is the signal the LLM uses to decide WHEN to call this tool. A vague docstring ('Do stuff with the web') leads to the LLM calling the tool at wrong times or not at all. A specific docstring ('Search the web for current facts, news, and statistics. Use when the user asks about recent events or data outside training.') leads to precise, correct tool selection.

bind_tools(): What Happens Internally

When you call `llm.bind_tools([my_tool])`, LangChain serialises each tool's Pydantic schema to JSON Schema format and includes it in the 'tools' parameter of the LLM API request. The LLM now knows these tools exist and can generate `tool_call` objects in its response that reference them by name with valid arguments.

■ Bug to avoid

The most common multi-agent bug: forgetting `bind_tools` on the researcher LLM.

```
researcher_llm = llm.bind_tools([web_search]) # CORRECT
```

```
researcher_llm = llm # WRONG: tool_calls always empty
```

Without `bind_tools`, the LLM cannot generate `tool_calls`.

Your search loop checks 'if resp.tool_calls' — which is always False.

Zero searches execute. `research_findings` stays None.

Output Parsers: Choosing the Right Tool

Output parsers transform raw LLM text into structured Python objects. The choice between them depends on how reliable you need the output to be and what Python object you need on the other side.

Parser	Returns	When to Use	Reliability
JsonOutputParser	Python dict	Flexible schema, any keys	Medium — depends on LLM following instr
PydanticOutputParser	Typed Pydantic object	Fixed schema, validated fields	High — Pydantic validates
<code>with_structured_output()</code>	Typed Pydantic object	Model supports function calling	Highest — model-level guarantee

For production routing and evaluation, always use `with_structured_output()`. It uses the model's native function-calling capability (when available) to guarantee the output matches the Pydantic schema. No regex, no

parsing code, no edge cases.

Callbacks: Observing Everything Without Changing Anything

LangChain's callback system is one of its most powerful but least-understood features. A callback handler is an object you attach to any LangChain call that gets notified of every event: LLM call start, LLM call end, tool call start, tool call end, chain start, chain end, errors. The critical property: you do not need to change your agent code at all. The callbacks observe from the outside.

In Cell 4.5 you build two production callbacks: a `TokenCounterCallback` that accumulates input and output tokens across every LLM call in a session, and a `LatencyTrackerCallback` that measures milliseconds per call. Attach them via `config={'callbacks': [counter, tracker]}`. In production, these feed dashboards that show cost-per-user and latency-per-agent.

Async: Why It Matters for Production

In a production web server, a single LLM call might take 2-5 seconds. If your server is synchronous, no other request can be served during those 5 seconds. Async changes this: while one request waits for the LLM, another request can be handled by the same Python process.

But async also helps in batch scenarios. `asyncio.gather()` runs multiple LLM calls simultaneously rather than sequentially. Three questions that each take 1 second: sequential = 3 seconds, concurrent with `asyncio.gather` = ~1 second. Every LangChain/LangGraph method has an async equivalent: `ainvoke()`, `astream()`, `astream_events()`. The async version of a graph is `graph.ainvoke()` and `graph.astream()`.

➡■ You now have all the components. Section 5 assembles them into a complete working agent with `create_react_agent`, then explores all four streaming modes including the token-level `astream_events` that powers real-time web UIs. We also introduce LangSmith Hub: version control for your prompts.

SECTION 5

create_react_agent + Streaming + LangSmith Hub

2:35 – 3:05 • 30 minutes • Cells 5.1 – 5.5

■ 2:35–3:05 • 30 min • Cells 5.1–5.5

The ReAct Pattern: Why It Works

ReAct (Reason + Act) is the dominant architecture for tool-using agents because it mirrors how a careful human researcher works: think about what you need, take an action, observe the result, think about what you learned, take the next action, repeat until done. The LLM is the thinking part. The tools are the acting part. LangGraph is the loop that connects them.

```
User query
↓
[agent node] LLM reasons: 'I need to search for X'      (REASON)
↓ tool_calls detected by conditional edge
[tools node] Executes web_search('X'), returns results (ACT)
↓ always loops back to agent
[agent node] LLM reasons with new information          (REASON)
↓ no more tool_calls – final answer ready
__end__
```

`create_react_agent` builds this entire graph for you in one factory call: the agent node (LLM with tools bound), the ToolNode (handles tool dispatch), the conditional routing (check for tool_calls after each LLM response), and the loop back (ToolNode always returns to agent). The `prompt=` parameter sets the system message. Note: always use `prompt=` in new code. The older `state_modifier=` parameter is deprecated.

astream_events v2: Token-Level Streaming for Web UIs

`astream_events` version 2 is the streaming mode that powers the ChatGPT experience. Instead of waiting for a complete node to finish before yielding, it fires a separate event for every single LLM output token, every tool call start, and every tool call end. Building a web UI that streams tokens to the browser in real time? This is the API you use.

astream_events v2: The Token Streaming Pattern

```
async for event in agent.astream_events({'messages': [HumanMessage(content='...')]}, version='v2'):
    if event['event'] == 'on_chat_model_stream':
        token = event['data']['chunk'].content
        print(token, end='', flush=True) # stream each token to UI
    elif event['event'] == 'on_tool_start':
        print(f"\n[SEARCHING: {event['name']}]") # show tool activity
    elif event['event'] == 'on_tool_end':
        print(f"[DONE: {str(event['data']['output'])[:80]}]")
```

Token Usage Tracking and Cost Calculation

Every `AIMessage` returned by a real LLM call has a `usage_metadata` attribute. It is a dict containing `input_tokens`, `output_tokens`, and `total_tokens`. Accumulating these across all messages in a pipeline run gives you the total cost of that run. Multiply by the provider's per-token pricing to get the dollar cost. LangSmith also captures this automatically per trace and per span, so you can see cost breakdowns without writing any tracking code — but tracking it yourself lets you surface it in your own dashboards.

LangSmith Hub: Version Control for Prompts

Prompts are code. They have versions, they change over time, they should be tested before deployment, and they should be revertable. LangSmith Hub gives prompts the same engineering treatment that code gets from git.

You push a `ChatPromptTemplate` to the Hub with a name. The Hub stores it with a version hash. You pull it anywhere with `client.pull_prompt('my-prompt')`. Pin to a specific version with `client.pull_prompt('my-prompt:abc123')`. Change a prompt without redeploying your application code. Compare two prompt versions on the same dataset. Roll back a bad prompt change in seconds.

➡■ Section 5 gave you a complete working agent. Section 6 adds the production layer that makes it survivable: structured tracing with `@traceable`, retry logic that handles real rate limits, graceful fallbacks when providers go down, and systematic evaluation that turns 'does this feel better?' into 'the score improved by 12%'.

SECTION 6

LangSmith + Production Patterns

3:05 – 3:30 • 25 minutes • Cells 6.1 – 6.5

■ 3:05–3:30 • 25 min • Cells 6.1–6.5

@traceable: Your Code in the Trace Tree

LangSmith automatically traces every LangChain and LangGraph call. But your pipeline is not just LangChain calls. There is preprocessing before the agent runs, postprocessing after it finishes, business logic in the middle. Without @traceable, these functions are invisible in the trace. With @traceable, they appear as named spans in the exact same tree as the LLM calls and tool executions.

LangSmith trace tree WITH @traceable:

```
full_pipeline          ← @traceable: root span (your function)
  preprocess_query      ← @traceable: child span (your function)
    latency: 52ms
  RunnableSequence      ← auto-traced: the agent
    agent (LLM call)     ← auto-traced: input/output/tokens/latency
    tools (Tavily search) ← auto-traced: query + results
    agent (LLM final answer) ← auto-traced
    postprocess_result   ← @traceable: child span (your function)
```

Everything in one place. One trace. Full picture.

Tags, Metadata, and Run Comparison

In production, you run thousands of traces per day. Without structure, finding a specific run is searching a haystack. Tags and metadata give you filters. Pass tags=['prompt-v2', 'experiment-A'] and metadata={'user_id': '123'} in the config dict of any invoke() call. In LangSmith, filter by tag to find all runs for a specific experiment. Select two experiments and click Compare to see side-by-side quality, latency, and cost differences.

Retry with Exponential Backoff: Not Optional in Production

HTTP 429 (rate limit exceeded) and 503 (service temporarily unavailable) are normal events in production, not exceptional ones. Every major LLM provider rate-limits. Every provider has occasional downtime. Without retry logic, a single blip fails your entire pipeline. With tenacity, you automatically wait and retry with increasing delays.

tenacity: Exponential Backoff Pattern

```
@retry(
    stop = stop_after_attempt(4),
    wait = wait_exponential(multiplier=1, min=1, max=30),
    reraise=True,
)
def resilient_call(question: str) -> str:
    return llm.invoke([HumanMessage(content=question)]).content
# Failure timeline: attempt 1 → fail → wait 1s
```



```
# attempt 2 → fail → wait 2s
# attempt 3 → fail → wait 4s
# attempt 4 → success (or reraise)
```

with_fallbacks(): One Line of Production Resilience

`with_fallbacks()` chains a primary LLM with one or more fallback LLMs. If the primary raises any exception — rate limit, timeout, quota, downtime — LangChain automatically tries the fallback. The caller receives a successful response. No special error handling code needed anywhere in your agent. Common pattern: expensive 70B model as primary, fast 8B model as fallback.

LangSmith Datasets and evaluate(): Engineering over Intuition

Without evaluation, improving your agent is based on intuition: 'the output looks better today'. With LangSmith evaluation, it is based on data: 'the keyword_coverage score improved from 0.64 to 0.91 after this prompt change'. This is the difference between amateur and professional agent development.

Build a dataset with `client.create_dataset()` and `client.create_example()`. Start with 3-5 examples. Add one for every production failure. Run `evaluate()` with evaluator functions that check specific quality dimensions. Store results as named experiments. Compare experiments side-by-side. Add to CI/CD: fail the pipeline if average score drops below threshold.

➡■ You have now mastered `create_react_agent` and all the production patterns that keep it running. But production systems often cannot use `create_react_agent` — they need custom graphs with precise control. Section 7 introduces three modern LangGraph features that give you that control: Command API, `Send()`, and Subgraphs.

SECTION 7

Custom Graphs + Command + Send() + Subgraphs + Testing

3:30 – 3:50 • 20 minutes • Cells 7.1 – 7.7

■ 3:30–3:50 • 20 min • Cells 7.1–7.7

When create_react_agent Is Not Enough

create_react_agent is excellent for the ReAct loop pattern. But the moment you need custom state fields, multiple specialised nodes beyond agent + tools, quality gates between steps, or precise control over which node runs when — you need a custom graph. Section 7 builds one: a 3-node research graph with a researcher, a tool executor, and a summariser. Every component from every previous section appears in it.

Node Unit Testing: The Practice That Saves Hours

The most important habit in LangGraph development is testing each node in isolation BEFORE wiring it into the graph. A node is a Python function. You can call it directly with a mock state and assert that its output dict has the right keys, the right types, and the right values. Tests that take 100ms catch bugs that would otherwise surface as 'the graph behaves strangely' after 10 minutes of execution.

■ Design principle

Test the router function before the graph.

Test each node function before the graph.

Test the full graph after all nodes pass in isolation.

Bugs found at node level cost 10x less to fix than bugs found at graph level.

The Command API: Modern Routing from Inside Nodes

The traditional pattern for routing in LangGraph is a separate conditional_edges() function that reads state and returns a node name. The Command API is the modern replacement: a node can return a Command object that bundles the state update AND the routing destination together. No separate routing function needed.

Why is this better? The routing logic lives inside the node that produced the output and understands it best. If you change what the LLM returns, you change the routing logic in the same place — not in a separate function elsewhere in the file. The type annotation Command[Literal['tools', '__end__']] also documents exactly which transitions are valid.

Command API: Routing from Inside the Node

```
def llm_node(state: MyState) -> Command[Literal['tools', '__end__']]:
    resp = llm_with_tools.invoke(state['messages'])
    if resp.tool_calls:
        return Command(update={'messages': [resp]}, goto='tools')
    return Command(update={'messages': [resp]}, goto='__end__')
# No add_conditional_edges() needed. Command handles routing.
```

Send(): Parallel Fan-Out at Graph Scale

Send() is the primitive for dispatching the same node to multiple inputs simultaneously. Where a normal edge sends one message to one node, Send() sends many messages to the same node in parallel. All dispatched calls run concurrently. Their outputs are merged by the field's reducer.

The pattern: a fan-out function returns a list of Send objects, one per input. Each Send names the target node and the input dict to pass. The graph runs all of them in parallel and waits for all to complete before moving to the next node.

Subgraphs: Composing Graphs as Nodes

A compiled LangGraph can be used as a node inside another graph. This is how complex systems are built: compose specialised subgraphs into a parent graph. The inner graph has its own State, its own nodes, its own logic. From the outside, it looks like a single node. Use get_graph(xray=1) in draw_mermaid_png() to see inside the subgraph nodes in the visualisation.

When to Use Each Advanced Feature: Decision Guide

Situation	Feature to Use	Why
Need routing from INSIDE the node (no separate API)	Separate API	Routing logic stays with the node that understands its output
Process N items simultaneously (parallel)	Send() fan-out	All N calls run at same time; reducer merges results
Reuse a complex subgraph as a single node	Subgraphs	Composability. Inner graph has its own state, nodes, logic.
Human needs to approve before a step executes	interrupt_before	Pause before node runs; human inspects inputs
Human needs to verify results before pipeline continues	interrupt_after	Pause after node runs; human inspects outputs
Agent output needs correction mid-pipeline	update_state()	Inject corrected fields; graph resumes with new state
Debug: 'step 9 failed, retry from step 5'	get_state_history()	Replay from any past checkpoint; original branch intact
Conversation too long for context window	RemoveMessage	Trim oldest messages; graph state shrinks automatically
Single server, persist across restarts	SqliteSaver	Drop-in replacement for MemorySaver; same agent code

These decision questions are the most common ones you face when designing a new graph. Print this table. Put it next to your keyboard. The answers do not change — the situations always map to the same features.

➡■ Section 7 gave you full control over graph architecture. Section 8 — the final section of Part 1 — adds the last three capabilities that separate prototypes from production systems: persistent memory that survives server restarts, context window management for long conversations, and the complete Human-in-the-Loop toolkit including time travel.

SECTION 8

Memory + HITL + Time Travel + SqliteSaver

3:50 – 4:00 • 10 minutes • Cells 8.1 – 8.6

■ 3:50–4:10 • 10+ min • Cells 8.1–8.6

The Graph Amnesia Problem

Every graph you have built so far resets every time you call `invoke()`. The conversation history, the research findings, the tool results — all gone when the function returns. Real agents need to remember. A customer support agent needs to recall what the user said three turns ago. A research agent needs to accumulate findings across multiple session. `MemorySaver` and `SqliteSaver` solve this by checkpointing state after every node, keyed by a `thread_id` that acts as the session identifier.

Checkpointner	Storage	Persists Restart?	Use Case
<code>MemorySaver</code>	RAM (in-process)	No	Development, testing, demos
<code>SqliteSaver</code>	SQLite file on disk	Yes	Single-server production
<code>AsyncPostgresSaver</code>	PostgreSQL	Yes	Multi-server production
<code>RedisSaver</code>	Redis	Yes	High-throughput production

■ Design principle

Agent code is IDENTICAL regardless of which checkpointner you use.
Switch from `MemorySaver` (development) to `SqliteSaver` (production) by changing exactly one argument in the `compile()` call.
This is by design. LangGraph separates logic from persistence.

RemoveMessage: Context Window Management Is Non-Optional

As a conversation grows, the messages list in state grows with it. Every LLM call includes the full history. At some point — usually 100 to 200 messages depending on the model — you exceed the context window and calls start failing. This is not an edge case. In any long-running production agent, it will happen. Plan for it from day one.

`RemoveMessage` solves this by deleting specific messages from state by their ID. The pattern: add a `trimmer_node` that runs BEFORE the chat node. The trimmer checks the message count. If it exceeds `KEEP_LAST_N`, it returns a dict of `RemoveMessage` objects for the oldest messages. LangGraph processes these deletions before the next node runs.

Human-in-the-Loop: `interrupt_before` vs `interrupt_after`

HITL is one of LangGraph's most distinctive features. The idea: pause the graph at a specific point, let a human read the state, optionally modify it with `update_state()`, then resume from exactly where it paused. Two interrupt modes handle different scenarios.

Mode	When it Pauses	Human Can...	Resume With
<code>interrupt_before=['node']</code>	BEFORE the node runs	Approve inputs, inject corrections before processing	<code>graph.stream(None, config)</code>
<code>interrupt_after=['node']</code>	AFTER the node runs	Inspect outputs, validate results before they propagate	<code>graph.state(None, config)</code>

The three-phase HITL flow: Phase 1 — `graph.stream(initial_input, config)` runs until the interrupt. Phase 2 — human calls `get_state(config)` to read current state, optionally calls `update_state(config, corrections)` to fix anything. Phase 3 — `graph.stream(None, config)` resumes from the saved checkpoint. Passing `None` as the input is the signal to continue from where it stopped.

Time Travel: Debugging at the Checkpoint Level

This is one of the most powerful debugging tools available to any software system, and LangGraph gives it to you for free. Every node activation saves a checkpoint. For a graph that runs 10 nodes, you have 10 checkpoints. `get_state_history()` returns all of them, most recent first.

The debugging workflow: your agent produces a bad result at step 9. You look at the checkpoint history and find the step 5 checkpoint. You call `invoke()` or `stream()` with that checkpoint's config. You are now running from step 5 — steps 1 to 4 are not re-executed. You can try a different prompt, a different tool, a different routing decision. Compare the results. The original timeline (steps 1-9) is still accessible. You created an alternate branch.

■ Instructor voice

Time travel is what makes LangGraph agent development feel like engineering rather than guesswork. When something breaks, you do not restart the entire expensive pipeline. You rewind to the moment before the mistake and try again.

Part 1 — Complete Feature Reference

Section	Feature	Notebook Cell
0	Provider-portability pattern: base_url swap	Cell 0.3
0	LangSmith auto-tracing: 2 env vars	Cell 0.2
1	All 5 message types + content formats	Cell 1.1
1	usage_metadata: token counting per call	Cell 1.2
1	Manual agent loop: feel the pain	Cell 1.3
1	7 failure modes: the case for LangGraph	Cell 1.4
2	State: add_messages reducer + custom reducers	Cell 2.1
2	State: multiple fields, Optional, Literal	Cell 2.2
2	Nodes: 4 patterns incl. ChatPromptTemplate	Cell 2.3–2.4
2	Edges: conditional routing + unit tests	Cell 2.5
2	Graph: Universal Assembly Pattern + visualise	Cell 2.6
2	Streaming: all 4 modes compared	Cell 2.7
2	update_state(): external state correction	Cell 2.8
3	ChatPromptTemplate + partial()	Cell 3.1
3	MessagesPlaceholder: conversation history	Cell 3.2
3	LCEL pipe operator + StrOutputParser	Cell 3.3
3	RunnableParallel + RunnableBranch	Cell 3.4
3	LCEL inside LangGraph nodes	Cell 3.5
4	@tool + Pydantic args_schema	Cell 4.1
4	bind_tools() internals: JSON Schema	Cell 4.2
4	ToolNode: handle_tool_errors	Cell 4.3
4	JsonOutputParser + PydanticOutputParser + with_structured_output	Cell 4.4
4	BaseCallbackHandler: token counter + latency tracker	Cell 4.5
4	Async: ainvoke, astream, asyncio.gather	Cell 4.6
5	create_react_agent: modern prompt= API	Cell 5.1
5	All 4 streaming modes on react_agent	Cell 5.2
5	astream_events v2: token-level streaming	Cell 5.3

5	Token usage tracking + cost estimation	Cell 5.4
5	LangSmith Hub: push/pull versioned prompts	Cell 5.5
6	@traceable: custom functions in trace tree	Cell 6.1
6	Tags + metadata + run comparison	Cell 6.2
6	tenacity: retry + exponential backoff	Cell 6.3
6	with_fallbacks(): graceful degradation	Cell 6.4
6	LangSmith datasets + evaluate()	Cell 6.5
7	Custom 3-node research graph	Cell 7.1
7	Router unit tests (zero LLM calls)	Cell 7.2
7	Stream-only execution pattern	Cell 7.3
7	Command API: routing from inside nodes	Cell 7.4
7	Send(): dynamic parallel fan-out	Cell 7.5
7	Subgraphs: graph-as-node + xray=1	Cell 7.6
7	Node unit testing pattern	Cell 7.7
8	MemorySaver: multi-turn + thread isolation	Cell 8.1
8	SqliteSaver: production persistence	Cell 8.2
8	RemoveMessage: context window management	Cell 8.3
8	HITL: interrupt_before + update_state()	Cell 8.4
8	interrupt_after: inspect tool results	Cell 8.5
8	Time travel: get_state_history + fork	Cell 8.6

The Bridge to Part 2: Everything You Built, at Scale

You have spent 4 hours building a complete toolkit. Every component is tested, every pattern is understood, every concept is clear. Part 2 takes all of it and assembles it into a single coherent system that does something genuinely useful in the real world: research a topic, write a report, have it reviewed, get human approval, and deliver the result as a real email to a real inbox.

Here is exactly how each Part 1 skill becomes a Part 2 component. Nothing in Part 2 is new. Everything has been practised here.

Part 1 Skill	Part 2 Agent / Feature
ChatPromptTemplate + MessagesPlaceholder	Researcher: research_prompt with topic + search_count injected from state
bind_tools() + ToolNode	Researcher: researcher_llm = llm_balanced.bind_tools([web_search])
LCEL chain: template llm	Writer: writer_template llm_fast (the complete writer chain)
with_structured_output(Pydantic)	Supervisor: RoutingDecision model. Reviewer: ReviewVerdict model.
@tool with Pydantic args_schema	web_search tool used by the Researcher in its internal search loop
tenacity retry + backoff	Reviewer LLM call: _call_reviewer() wrapped with @retry(stop=3)
with_fallbacks()	Every agent LLM: llm_strong.with_fallbacks([llm_fast])
BaseCallbackHandler	AgentStepLogger: tracks latency and tokens per agent step
async ainvoke()	Email Notifier: _format_html_async() calls llm_fast.ainvoke()
interrupt_before	Human approval gate: interrupt_before=['notifier']
update_state()	Human correction: inject revised draft before email is sent
get_state_history()	Debug and replay from any checkpoint in the 5-agent run
LangSmith evaluate()	Part 2 Section 8: 4 evaluators on a dataset of research examples
Node unit testing	Each of the 5 agents tested in isolation before wiring into graph

“The 5-agent system is not a new thing you learn. It is every Part 1 concept applied simultaneously to a real problem. You already have everything you need.”